

β -DQN: Diverse Exploration via Learning a Behavior Function

Hongming Zhang

2023.10.06

Efficient exploration remains a pivotal challenge in reinforcement learning (RL). While numerous methods have been proposed, their lack of [simplicity](#), [generality](#) and [computational efficiency](#) often lead researchers to choose simple techniques such as ϵ -greedy.

We propose β -DQN to improve exploration by constructing a set of [diverse policies](#) through a [behavior function \$\beta\$](#) learned from the replay memory.

- [Straightforward to implement](#)
- Imposes [minimal hyper-parameter tuning](#) demands
- Adds a [modest computational overhead](#) to DQN
- [Enhances performance](#) and [exhibits broad applicability](#) in both easy and hard exploration domains

Motivation



Simple method such as ϵ -greedy:

- State-independent
- Lack of temporal persistence

Complex method such as RND:

- More hyper-parameters to tune
- More computational cost
- Not general

Our goal:

Design an **effective** method while maintaining **simplicity** and **generality**.

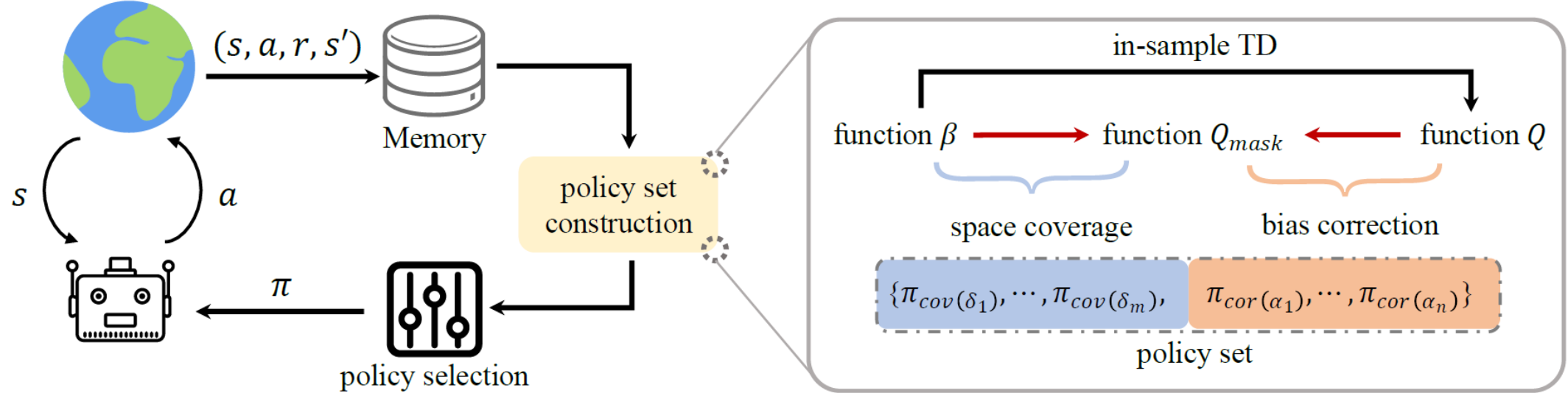


Figure 1: An overview of our method. We learn a behavior function β from replay memory, constrain the TD learning to in-sample data to learn function Q , and mask Q with β to get Q_{mask} . With β , Q and Q_{mask} , we construct a set of policies ranging from exploration for space coverage and bias correction to pure exploitation. A meta-controller is designed to adaptively select a policy to interact with then environment for each episode.

Learning Basic Functions

- Behavior function β

$$\mathcal{L}_\beta = -\frac{1}{|\mathcal{B}|} \sum_{(s,a) \in \mathcal{B}} \log \beta(s, a)$$

The β can differentiate between **actions are frequently taken** and those are **rarely taken** at a state.

- Action value function Q

$$Q(s, a) \leftarrow r + \gamma \max_{a': \beta(a'|s') > \epsilon} Q(s', a')$$

Because the data coverage in the replay memory is limited to a tiny subset of the whole environment space, $\pi := \operatorname{argmax}_a Q(s, a)$, is an **optimistic policy** that may take erroneously overestimated actions.

- Conservative action value function Q_{mask}

$$Q_{mask}(s, a) = \begin{cases} Q(s, a), & \text{if } \beta(s, a) > \epsilon \\ \min_{a \in \mathcal{A}} Q(s, a), & \text{otherwise} \end{cases}$$

$\pi := \operatorname{argmax}_a Q_{mask}(s, a)$ is a pure **exploitation policy**.

$\epsilon = 0.05$ is a small number.

Constructing Policy Set

- Exploration for **Space Coverage**

$$\pi_{cov(\delta)} = \begin{cases} \underset{a}{\operatorname{argmax}} Q_{mask}(s, a), & \text{if } \beta(s, a) > \delta \forall a \in \mathcal{A} \\ \text{DiscreteU}(\{a: \beta(s, a) \leq \delta\}), & \text{otherwise} \end{cases}$$

$\delta \in [0,1]$ is a parameter. $\pi_{cov(0)}$ is the pure exploitation policy following Q_{mask} , $\pi_{cov(1)}$ is a random policy.

- Exploration for **Bias Correction**

$$\pi_{cor(\alpha)} = \underset{a}{\operatorname{argmax}} (\alpha Q(s, a) + (1 - \alpha) Q_{mask}(s, a))$$

$\alpha \in [0,1]$ is a parameter. $\pi_{cor(0)}$ is the pure exploitation policy following Q_{mask} , $\pi_{cor(1)}$ is the optimistic policy following Q .

- Constructing **Policy Set**

$$\Pi = \{\pi_{cov(\delta_1)}, \dots, \pi_{cov(\delta_m)}, \pi_{cor(\alpha_1)}, \dots, \pi_{cor(\alpha_n)}\}$$

The policy set **does not inject specialized inductive bias** thus is general for a wide range of tasks, and the computation will not increase when we add more policies with different δ and α .

Meta-Controller for Policy Selection



Consider the policy selection problem as a **non-stationary multi-armed bandit** (MAB)

- Policy set: $\Pi = \{\pi_0, \dots, \pi_{N-1}\}$ with N policies
- Sliding-window length: L
- Episode return at episode m when choose π_i : $R_m(A_m = \pi_i)$

We normalize the return into $[0,1]$ by $\frac{R_m - R_{\min}}{R_{\max} - R_{\min}}$, where $R_{\max}$ and $R_{\min}$ are the maximum and minimum return in the sliding window

- The number of times policy π_i has been selected after k episodes: $N_k(\pi_i, L) = \sum_{m=0 \vee k-L}^{k-1} \mathbb{I}(A_m = \pi_i)$

$0 \vee k - L$ means $\max(0, k - L)$, $\mathbb{I}(\cdot)$ is indicator function $\mathbb{I}(U) = \begin{cases} 1, & \text{if } U \text{ is True} \\ 0, & \text{otherwise} \end{cases}$

Meta-Controller for Policy Selection



- Empirical **mean return** of π_i until episode k : $\mu_k(\pi_i, L) = \frac{1}{N_k(\pi_i, L)} \sum_{m=0 \vee k-L}^{k-1} R_m(\pi_i) \mathbb{I}(A_m = \pi_i)$
- Exploration bonus at episode m when choose π_i : $B_m(\pi_i) = \frac{1}{T_m} \sum_{t=0}^{T_m-1} \mathbb{I}(\pi_i(s_t) \neq \arg\max_a Q_{mask}(s_t, a))$
- Empirical **mean bonus** of π_i until episode k : $b_k(\pi_i, L) = \frac{1}{N_k(\pi_i, L)} \sum_{m=0 \vee k-L}^{k-1} B_m(\pi_i) \mathbb{I}(A_m = \pi_i)$

Then for episode k , we select a policy by

$$A_k = \begin{cases} \pi_i, & \text{if } N_k(\pi_i, L) = 0 \\ \arg\max_{\pi_i} (\mu_k(\pi_i, L) + b_k(\pi_i, L)), & \text{otherwise} \end{cases}$$

Pseudo-code

Qmask

Algorithm 1 β -DQN

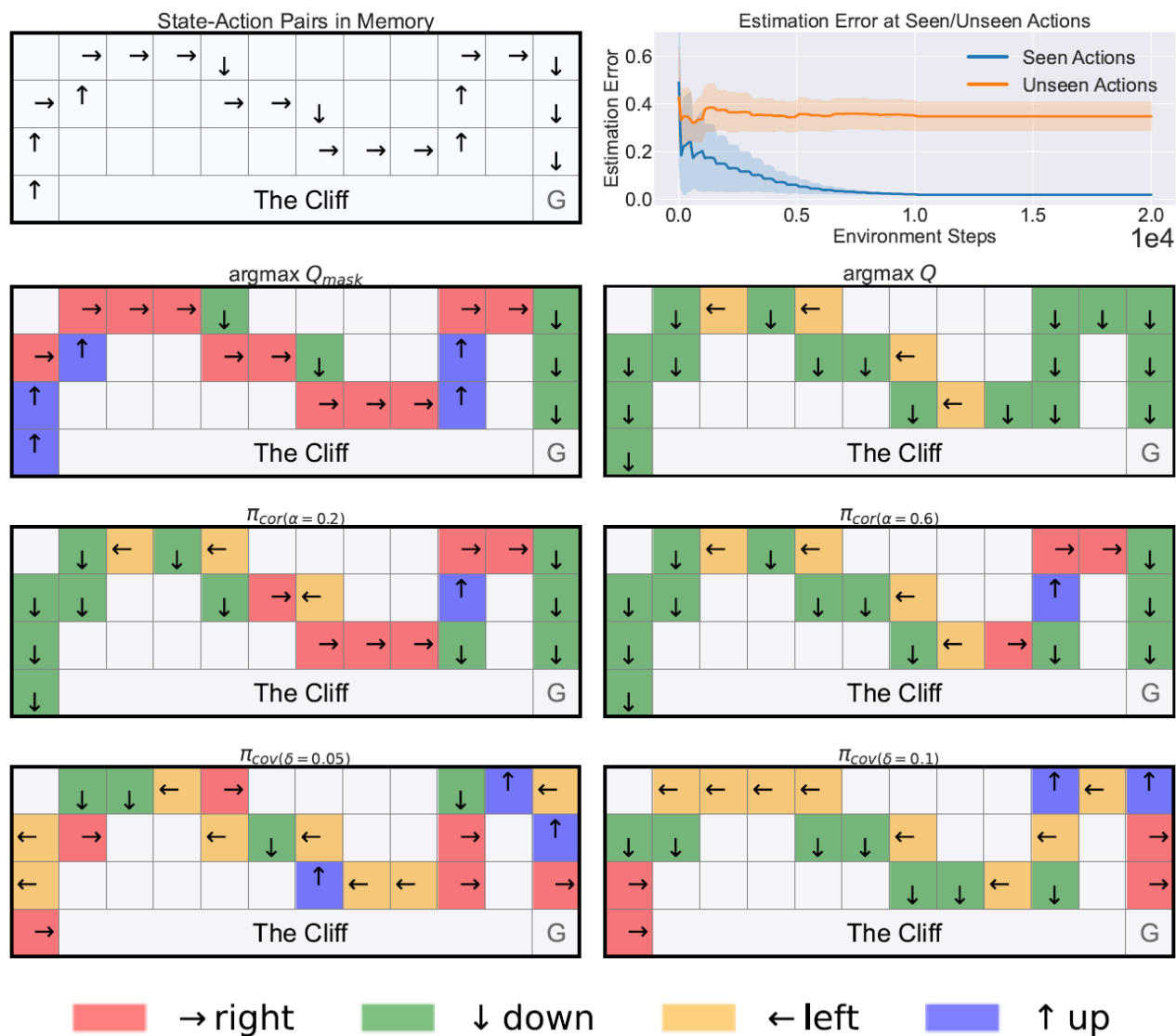
- 1: Initialize replay memory $\mathcal{D}$ with fixed size
 - 2: Initialize functions β, Q, Q_{mask} , and construct policy set Π following Eqs. (4) to (6)
 - 3: **for** episode $k = 0$ **to** K **do**
 - 4: Select a policy π according to Eq. (12)
 - 5: Initialize the environment $s_0 \leftarrow Env$
 - 6: **for** environments step $t = 0$ **to** T **do**
 - 7: Select an action $a_t \sim \pi(\cdot|s_t)$
 - 8: Execute a_t in Env and get r_t, s_{t+1}
 - 9: Store transition (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$
 - 10: Update β and Q following Eqs. (1) and (2)
 - 11: **end for**
 - 12: **end for**
-

$$\pi_{cov(\delta)} = \begin{cases} \underset{a}{\operatorname{argmax}} Q_{mask}(s, a), & \text{if } \beta(s, a) > \delta \forall a \in \mathcal{A} \\ \text{DiscreteU}(\{a: \beta(s, a) \leq \delta\}), & \text{otherwise} \end{cases}$$
$$\pi_{cor(\alpha)} = \underset{a}{\operatorname{argmax}} (\alpha Q(s, a) + (1 - \alpha) Q_{mask}(s, a))$$
$$\Pi = \{\pi_{cov(\delta_1)}, \dots, \pi_{cov(\delta_m)}, \pi_{cor(\alpha_1)}, \dots, \pi_{cor(\alpha_n)}\}$$

$$A_k = \begin{cases} \pi_i, & \text{if } N_k(\pi_i, L) = 0 \\ \underset{\pi_i}{\operatorname{argmax}} (\mu_k(\pi_i, L) + b_k(\pi_i, L)), & \text{otherwise} \end{cases}$$

$$\mathcal{L}_\beta = -\frac{1}{|\mathcal{B}|} \sum_{(s,a) \in \mathcal{B}} \log \beta(s, a)$$
$$Q(s, a) \leftarrow r + \gamma \max_{a': \beta(a'|s') > \epsilon} Q(s', a')$$

Experiments – Toy Example



Update times

Figure 2: Policy diversity at a specific case. Top left shows the state-action pairs in the memory. Top right shows value errors at seen and unseen actions, implying different roles of Q and Q_{mask} . Remaining rows show diverse policies derived from β , Q and Q_{mask} . These policies take different actions and lead to different states, which benefits the learning.

Experiments – Toy Example

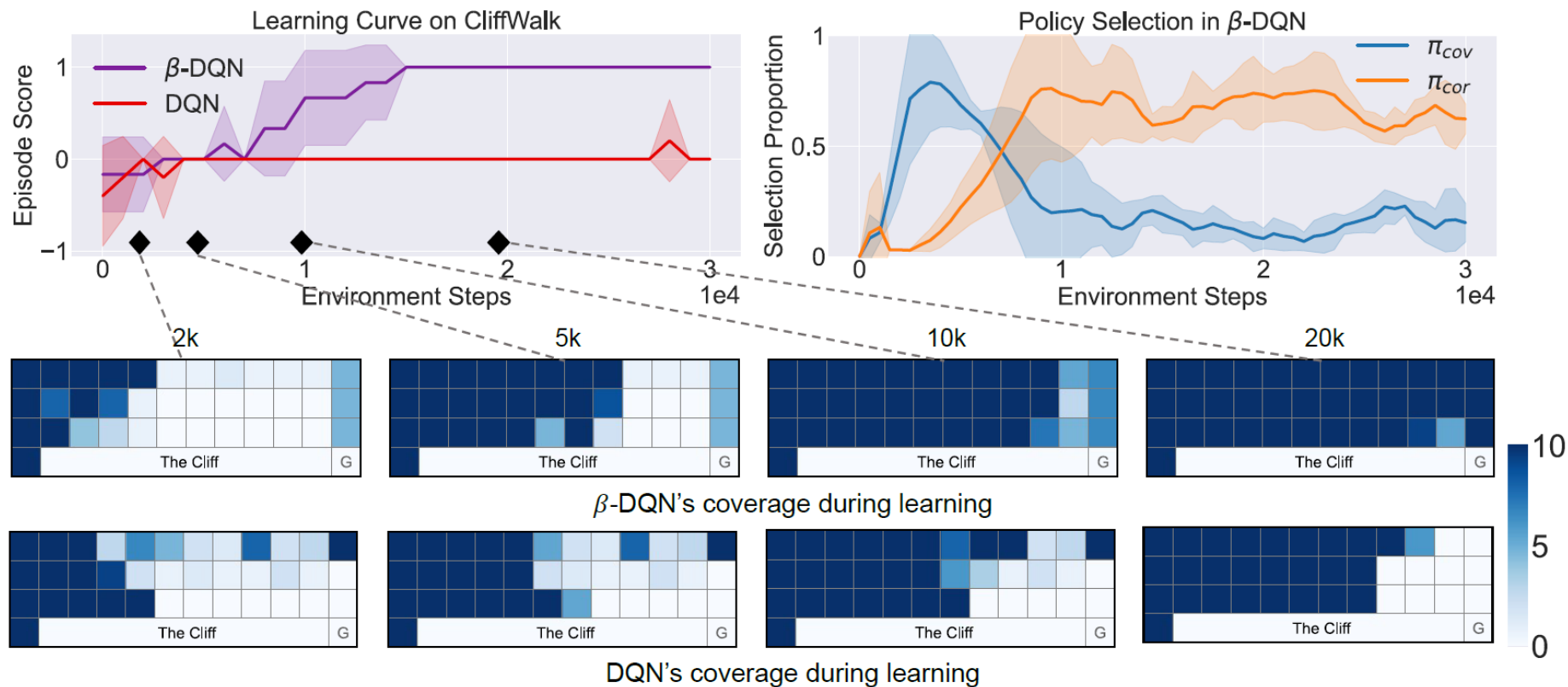


Figure 3: Details during the learning process. The top left shows the learning curves of β -DQN and DQN. The heatmaps show the state coverage during learning. Our method explores the whole space quickly and finds the path to the goal state which gives reward 1. In contrast, DQN agent learns how to avoid falling into the cliff but fails to reach the goal state G. The top right figure further shows the policy selection in β -DQN. It first tries to explore the state space with π_{cov} , and then converts to correct inaccurate estimation using π_{cor} .

Experiments – Overall Performance



Table 1: Overall performance on MiniGrid (success rate) and MinAtar (final score). Numbers in bold represent the method that achieves the best performance. The last row shows the computational overhead compared with DQN. Our method achieves best performance on most games and adds mild computational overhead.

Environment		DQN	BootDQN	ϵ z-greedy	RND	LESSON	β -DQN (Ours)
MiniGrid	DoorKey	0.44	0.11	0.0	0.99	0.86	0.98
	Unlock	0.22	0.17	0.0	0.95	0.64	0.99
	RedBlueDoors	0.43	0.46	0.0	1.0	0.73	0.38
	SimpleCrossing-Easy	1.0	1.0	0.95	0.95	0.97	0.99
	SimpleCrossing-Hard	1.0	0.81	0.05	0.93	0.6	1.0
	LavaCrossing-Easy	0.29	0.66	0.26	0.68	0.75	0.84
	LavaCrossing-Hard	0.0	0.01	0.0	0.39	0.06	0.16
MinAtar	Asterix	22.78	22.54	18.79	13.4	18.43	39.09
	Breakout	16.69	21.88	19.06	14.1	17.71	29.04
	Freeway	60.78	59.94	59.68	49.26	54.38	62.56
	Seaquest	14.66	14.31	16.98	5.61	9.41	33.23
	SpaceInvaders	67.28	69.91	68.7	31.58	55.94	98.28
Computational Overhead		100%	195.34 %	94.32 %	152.57 %	371.07 %	138.78 %

$$\Pi = \{\pi_{cov(0.05)}, \pi_{cov(0.1)}, \pi_{cor(0)}, \pi_{cor(0.1)}, \pi_{cor(0.2)}, \dots, \pi_{cor(1)}\}, L = 1000$$

Experiments – Ablations

The role of the two kinds of exploration polices:

$$\pi_{cov} = \{\pi_{cov}(0.05), \pi_{cov}(0.1)\}$$

$$\pi_{cor} = \{\pi_{cor}(0.1), \pi_{cor}(0.2), \dots, \pi_{cor}(1)\}$$

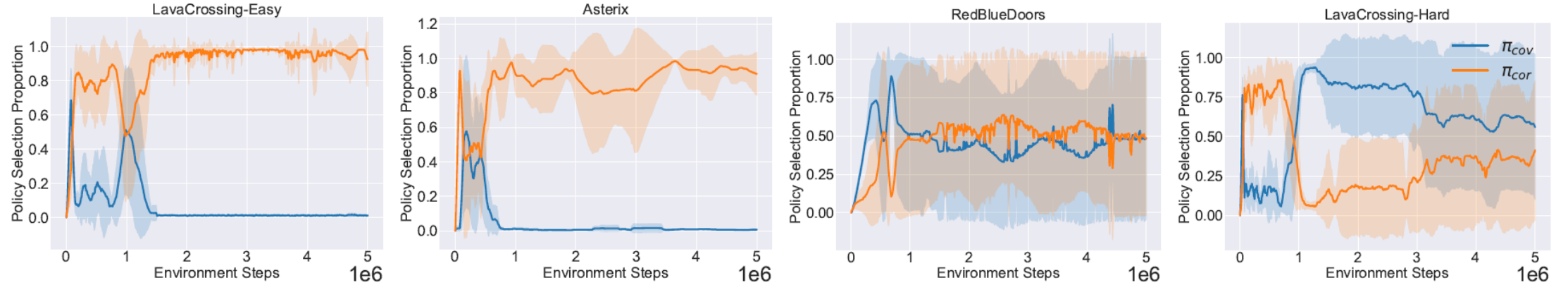


Figure 4: Policy selection during learning. In simple (LavaCrossing-Easy) or dense reward (Asterix) environments, exploring for bias correction plays more important role. In hard exploration environments (RedBlueDoors, LavaCrossing-Hard), the two kinds of policies interleave and result in a more intricate selection pattern.

Experiments – Ablations

The performance of policies in the policy set:

$\operatorname{argmin}_a \beta$ takes actions with the least probability thus learns nothing

$\operatorname{argmax}_a Q_{mask}$ chooses in-sample greedy actions and performs the best

$\operatorname{argmax}_a Q$ takes greedy actions among the whole action space and may take overestimated actions

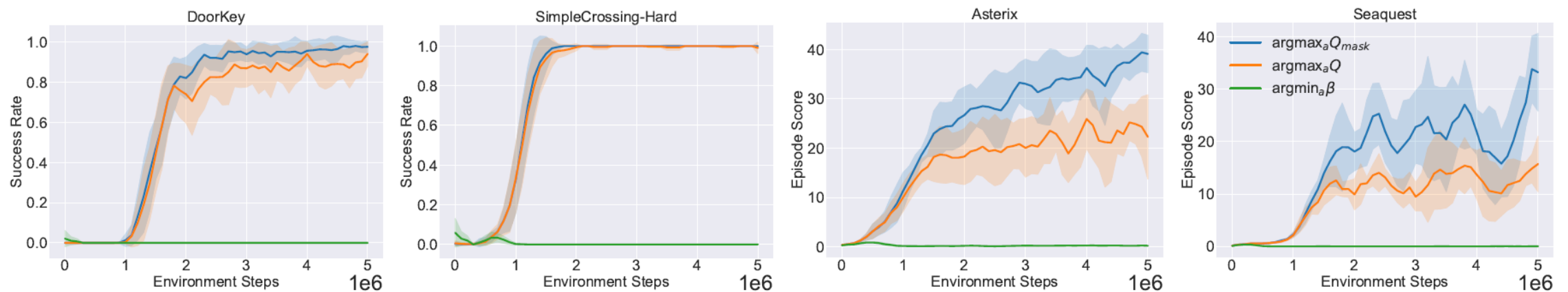


Figure 5: The performance of the three basic policies during learning. $\operatorname{argmin}_a \beta$ learns nothing since it does not care about performance. $\operatorname{argmax}_a Q_{mask}$ chooses in-sample greedy actions and performs the best. $\operatorname{argmax}_a Q$ takes greedy actions among the whole action space and may take overestimated actions.

Experiments – Ablations

Learning two separate Q :

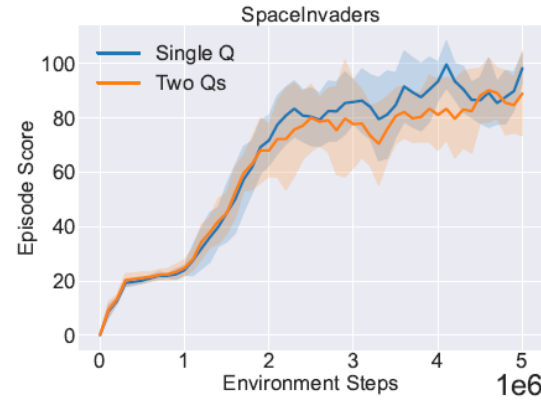
$$Q(s, a) \leftarrow r + \gamma \max_{a': \beta(a'|s') > \epsilon} Q(s', a')$$
$$Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a')$$

Size of policy set:

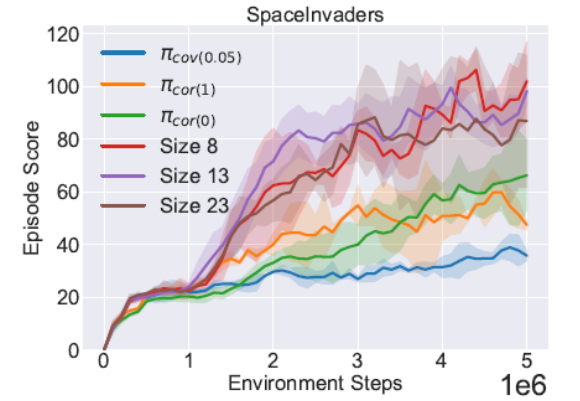
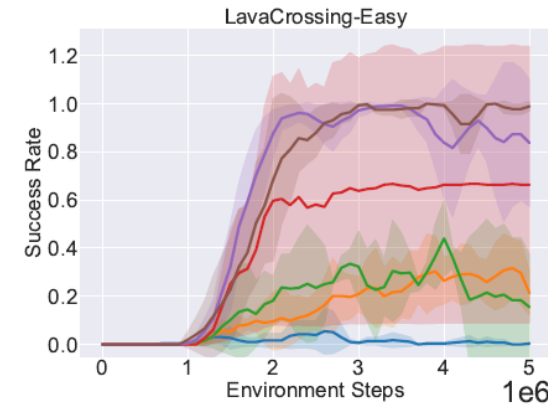
Size 8: $\{\pi_{cov(0.05)}, \pi_{cov(0.1)}, \pi_{cor(0)}, \pi_{cor(0.2)}, \pi_{cor(0.4)}, \dots, \pi_{cor(1)}\}$

Size 13: $\{\pi_{cov(0.05)}, \pi_{cov(0.1)}, \pi_{cor(0)}, \pi_{cor(0.1)}, \pi_{cor(0.2)}, \dots, \pi_{cor(1)}\}$

Size 23: $\{\pi_{cov(0.05)}, \pi_{cov(0.1)}, \pi_{cor(0)}, \pi_{cor(0.05)}, \pi_{cor(0.1)}, \dots, \pi_{cor(1)}\}$



(a) Learning two separate Q s.



(b) The influence of the policy set size.

Figure 6: Ablation studies on different configurations. (a) We learn two Q s separately and find no obvious difference, which means it is enough to derive two Q s from Eq. (2). (b) We construct policy sets with different sizes. There is clear performance improvement when we include all the three functions in the policy set.

Conclusion



In this paper, we improve exploration by constructing a group of diverse policies via only additionally [learning a behavior function \$\beta\$](#) from the replay memory with supervised learning.

- With β , we [construct a set of policies](#) ranging from exploration for space coverage and bias correction to pure exploitation.
- Then an adaptive [meta-controller](#) is designed to select an effective policy to interact with the environment for each episode.

Our method is [simple, general](#) and [adds little computational overhead](#) to DQN.

Experiments conducted on MinAtar and MiniGrid demonstrate our method is effective and exhibits broad applicability in both easy and hard exploration domains. Future work could be extending our method to environments with continuous action space.



THANKS!